

Rozmowa OCPP krok po kroku

Jedna sesja ładowania Q11 przez most OCPP, ramka po ramce · AMPERE POINT · 26 września 2026
· oprac. Dawid Cekała

Kto rozmawia i jakim językiem

Uczestnik	Rola	Z kim rozmawia	Łącze i język
serwer OCPP, CSMS	system operatora: karty, rozliczenia, zdalne sterowanie	tylko z mostem	WebSocket, OCPP 1.6J
most OCPP	udaje wobec serwera ładowarkę Q11 - 543204516334	z serwerem i z DevKitem	dwa połączenia WebSocket, dwa języki
DevKit	moduł Shelly w Q11	z mostem i ze sterownikiem Q11	WebSocket z poleceniami Shelly; łącze szeregowe z protokołem Tuya
sterownik Q11	ładuje auto	tylko z DevKitem	łącze szeregowe 9600 b/s, protokół Tuya

Serwer nie wie, że po drugiej stronie jest most, a nie ładowarka. Sterownik Q11 nie wie nic o OCPP.

Oznaczenia w dokumencie:

-  **prawdziwa ramka** skopiowana z logów z 25.09.2026,
-  **przykład:** tak zbuduje ją nasz most według swojego kodu, ale tej ramki jeszcze nie zaobserwowaliśmy, bo pełna transakcja z ładowaniem czeka na test z symulatorem.

Ramki OCPP idą łączyłem w jednej linii. W dokumencie są rozpisane na kilka linii, żeby dało się je czytać.

Jak czytać ramkę OCPP

Każda wiadomość OCPP to tablica JSON. Pierwsza liczba mówi, czym jest wiadomość:

Pierwsza liczba	Nazwa w OCPP	Po polsku	Budowa
2	CALL	zapytanie albo polecenie	[2, "numer", "Akcja", {dane}]
3	CALLRESULT	odpowiedź	[3, "numer", {dane}]
4	CALLERROR	odpowiedź „nie da się”	[4, "numer", "kod błędu", "opis", {szczegóły}]

- **Numer** wymyśla ten, kto zadaje pytanie. Nasz most i nasz serwer używają losowych identyfikatorów UUID. Odpowiedź powtarza ten sam numer, więc wiadomo, na co odpowiada.
- **Akcja** jest tylko w zapytaniu. Odpowiedź jej nie powtarza, bo wynika z numeru.

- **Zapytanie może wysłać każda strona.** Ładowarka pyta o rejestrację i zgłasza pomiary, a serwer wysyła polecenia. Kierunek łącza nie ogranicza, kto pyta.
- **Jedno pytanie naraz.** Strona nie wysyła kolejnego zapytania, dopóki nie dostanie odpowiedzi na poprzednie albo nie minie limit czasu.

Krok 0. Otwarcie łącza

Most → **serwer OCPP** · **zwykłe połączenie sieciowe, które przechodzi w WebSocket** · **zapytanie i odpowiedź**
 Łącze zawsze otwiera ładowarka, u nas most. Zaczyna od zwykłego zapytania WWW z prośbą o przełączenie na WebSocket:

```
GET /Q11-543204516334 HTTP/1.1
Host: localhost:9000
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Version: 13
Sec-WebSocket-Key: <losowy klucz>
Sec-WebSocket-Protocol: ocpp1.6
```

Serwer się zgadza:

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: <skrót klucza>
Sec-WebSocket-Protocol: ocpp1.6
```

- /Q11-543204516334 na końcu adresu to **identyfikator ładowarki**. Po nim serwer wie, kto się połączył.
- Sec-WebSocket-Protocol: ocpp1.6 to deklaracja języka. Serwer potwierdza, że będzie mówić OCPP 1.6.
- 101 Switching Protocols oznacza zgodę. Od tej chwili to samo połączenie jest stałą linią w obie strony.
- Tak wygląda standardowe otwarcie WebSocket, które robi biblioteka. Nie zapisywaliśmy tych nagłówków, ale serwer zapisał jego wynik: „POLACZENIE ładowarki Q11-543204516334, podprotokół ocpp1.6”, ✓ 25.09, 11:56:07.
- W produkcji adres zaczyna się od wss://, a ładowarka podaje hasło albo certyfikat.

Krok 1. Przedstawienie się: BootNotification

Most → **serwer** · **zapytanie** · ✓ **prawdziwe, 11:56:07**

```
[2, "aa145e23-b8f2-4fb9-9caa-dabe46698938", "BootNotification",
  {"chargePointModel": "Q11",
   "chargePointVendor": "AmperePoint",
   "chargePointSerialNumber": "Q11-543204516334",
   "firmwareVersion": "Shelly X DevKit + skrypt v3"}]
```

Serwer → **most** · **odpowiedź** · ✓ **prawdziwe**

```
[3, "aa145e23-b8f2-4fb9-9caa-dabe46698938",
  {"currentTime": "2026-09-25T09:56:07Z", "interval": 60, "status": "Accepted"}]
```

- Ładowarka mówi, kim jest: producent, model, numer seryjny, wersja oprogramowania.
- "status": "Accepted" to zgoda na pracę. Przy „Pending” albo „Rejected” ładowarka nie może jeszcze nic innego wysyłać.
- "interval": 60 to polecenie: „dawaj znak życia co 60 s”.
- "currentTime" to czas serwera w UTC. 09:56:07Z to 11:56:07 czasu polskiego.

- Gdzie są DevKit i Q11? Nigdzie. Rejestracja to sprawa tylko między mostem a serwerem.

Krok 2. Gotowość: StatusNotification

Most → serwer · zapytanie · ✓ prawdziwe, 11:56:07

```
[2, "55c8d3e7-b27e-4311-b681-80156eb2e953", "StatusNotification",
  {"connectorId": 1, "errorCode": "NoError", "status": "Available",
   "timestamp": "2026-09-25T09:56:07Z", "info": "Q11: charger_free"}]
```

Serwer → most · odpowiedź · ✓ prawdziwe

```
[3, "55c8d3e7-b27e-4311-b681-80156eb2e953", {}]
```

- "connectorId": 1 to jedyne złącze Q11. Numer 0 oznaczałby całą ładowarkę.
- "Available" wziął się ze stanu Q11 charger_free, czyli wolna. Most przepisuje stan Q11 na słowo OCPP.
- "info" to dowolny opis. Wpisujemy tam oryginalny stan Q11, żeby było widać, skąd się wziął.
- Pusta odpowiedź {} znaczy „przyjąłem”. Na zgłoszenie stanu serwer nie ma nic do dodania.

Krok 3. Znak życia: Heartbeat

Most → serwer · zapytanie · ✓ prawdziwe, 13:53:40

```
[2, "2ad66794-3b62-4532-b95b-cb8347df4767", "Heartbeat", {}]
```

Serwer → most · odpowiedź · ✓ prawdziwe

```
[3, "2ad66794-3b62-4532-b95b-cb8347df4767", {"currentTime": "2026-09-25T11:53:40Z"}]
```

- Zapytanie nie niesie danych, sama jego obecność znaczy „żyję”. Odpowiedź przy okazji podaje czas serwera.
- Od 11:57 do 13:41 most wysłał 104 takie zapytania, co 60 s, jak kazał serwer w kroku 1.

Krok 4. Zdalny start: RemoteStartTransaction

Serwer → most · zapytanie, czyli polecenie · ✓ prawdziwe, 11:57:20

```
[2, "ed769d84-e5f6-409a-9537-ce29b6d5134d", "RemoteStartTransaction",
  {"idTag": "KARTA-TEST", "connectorId": 1}]
```

Most → serwer · odpowiedź · ✓ prawdziwe

```
[3, "ed769d84-e5f6-409a-9537-ce29b6d5134d", {"status": "Accepted"}]
```

- Teraz pyta serwer: „uruchom ładowanie dla karty KARTA-TEST”. Ta sama linia działa w drugą stronę.
- "idTag" to identyfikator użytkownika. Q11 nie ma czytnika kart, więc most tylko zapamiętuje tę wartość i wstawi ją do StartTransaction.
- "Accepted" znaczy „przyjąłem polecenie”, a nie „ładowanie trwa”. Ładowanie zacznie się, gdy auto będzie gotowe.

Co most robi dalej, na łączu z DevKitem. Wysyła polecenie Shelly „włącz ładowanie”:

```
{"id": 9, "src": "most-ocpp", "method": "Boolean.Set", "params": {"id": 200, "value": true}}
```

🔴 Numer id nadaje most. W teście o 11:57:20 ładowanie było już włączone, więc pole się nie zmieniło i DevKit nic nie wysłał do Q11. Gdy ładowanie jest wyłączone, DevKit wysłał do Q11 taką ramkę Tuya, ✅ prawdziwa z 10:52:37, wtedy z aplikacji:

```
DevKit → Q11: 55 AA 00 06 00 05 12 01 00 01 01 1F   ustaw punkt danych 18 na „tak”
Q11 → DevKit: 55 AA 03 07 00 05 12 01 00 01 01 23   punkt danych 18 = „tak”
```

Budowa: 55 AA początek, 00 albo 03 wersja, 06 ustaw albo 07 melduję, 00 05 długość danych, 12 punkt danych 18, 01 typ „tak lub nie”, 00 01 długość wartości, 01 wartość, na końcu suma kontrolna.

Krok 5. Auto gotowe, ładowanie rusza: StatusNotification i StartTransaction

Tu rozmowa zaczyna się w Q11, a nie na serwerze.

Q11 → DevKit · łączy szeregowo · meldunek · 🔴 przykład

```
55 AA 03 07 00 05 03 04 00 01 01 17   punkt danych 3 = pozycja 1 = auto podłączone
55 AA 03 07 00 05 03 04 00 01 04 1A   punkt danych 3 = pozycja 4 = ładuje
```

DevKit → most · WebSocket · powiadomienie Shelly · 🔴 przykład

```
{"src": "apq11dev-543204516334", "dst": "most-ocpp", "method": "NotifyStatus",
 "params": {"ts": 1790400000.00, "enum:200": {"value": "charger_charging"}}
```

Skrypt usługi w DevKicie przetłumaczył pozycję 4 na nazwę charger_charging. Powiadomienie to nie jest zapytanie: nie ma numeru id i nie wymaga odpowiedzi.

Most → serwer · dwa zapytania · 🔴 przykład

```
[2, "b71c...", "StatusNotification",
 {"connectorId": 1, "errorCode": "NoError", "status": "Charging",
  "timestamp": "2026-09-26T10:00:00Z", "info": "Q11: charger_charging"}]

[2, "c3e4...", "StartTransaction",
 {"connectorId": 1, "idTag": "KARTA-TEST", "meterStart": 1234560,
  "timestamp": "2026-09-26T10:00:00Z"}]
```

Serwer → most · odpowiedź na StartTransaction · 🔴 przykład

```
[3, "c3e4...", {"transactionId": 1, "idTagInfo": {"status": "Accepted"}}]
```

- Zanim ruszyło ładowanie, most wysłał StatusNotification „Preparing”, gdy Q11 zgłosiła „auto podłączone”.
- "meterStart": 1234560 to stan licznika w Wh w chwili startu, czyli 1234,56 kWh z punktu danych 1.
- "transactionId": 1 nadaje serwer. Most musi go zapamiętać, bo każdy pomiar i koniec transakcji będą się do niego odwoływać.
- "idTag" pochodzi z kroku 4. Bez zdalnego startu most wpisałby Q11-LOKALNIE.

Krok 6. Pomiary: MeterValues

Most → serwer · zapytanie · 🔴 przykład, 12 minut po starcie, 10 A na fazę

```
[2, "d9a0...", "MeterValues",
 {"connectorId": 1, "transactionId": 1,
  "meterValue": [{"timestamp": "2026-09-26T10:12:00Z",
    "sampledValue": [
      {"value": "1235940", "measurand": "Energy.Active.Import.Register", "unit": "Wh"},
      {"value": "6900", "measurand": "Power.Active.Import", "unit": "W"}],
    }]
```

```
{ "value": "10.0", "measurand": "Current.Import", "phase": "L1", "unit": "A"},
{ "value": "230.1", "measurand": "Voltage", "phase": "L1", "unit": "V"},
{ "value": "10", "measurand": "Current.Offered", "unit": "A"} ] ] ] }
```

Serwer → most · odpowiedź

```
[3, "d9a0...", {}]
```

- Wartości są tekstem, taki jest wymóg OCPP.
- Licznik wzrósł z 1 234 560 do 1 235 940 Wh, czyli o 1,38 kWh. Tyle daje 6,9 kW przez 12 minut: $6,9 \times 0,2 \text{ h} = 1,38 \text{ kWh}$.
- Prąd i napięcie idą osobno dla L1, L2 i L3. W przykładzie pokazana jest tylko L1.
- Most wysyła pomiary co 30 s w trakcie transakcji. Serwer może też poprosić o nie od razu przez Trigger-Message. Tak było w teście o 11:56:48, ✓, wtedy z samymi zerami, bo auto było odłączone.

Krok 7. Zmiana limitu prądu: SetChargingProfile

To jedyny krok, który przeszedł w teście przez cały łańcuch, od serwera do Q11 i z powrotem.

Serwer → most · polecenie · ✓ prawdziwe, 11:56:48

```
[2, "60ca351b-c144-4e5a-b88d-b9a9e7d01a4b", "SetChargingProfile",
{ "connectorId": 1,
  "csChargingProfiles": { "chargingProfileId": 1, "stackLevel": 0,
    "chargingProfilePurpose": "TxDefaultProfile", "chargingProfileKind": "Absolute",
    "chargingSchedule": { "chargingRateUnit": "A",
      "chargingSchedulePeriod": [ { "startPeriod": 0, "limit": 10.0 } ] } } } ] }
```

Most → DevKit · polecenie Shelly · ✓ prawdziwe, ta sama sekunda

```
{ "id": 8, "src": "most-ocpp", "method": "Number.Set", "params": { "id": 200, "value": 10 } }
```

DevKit → most · odpowiedź Shelly · ✓ prawdziwe

```
{ "id": 8, "src": "apq11dev-543204516334", "dst": "most-ocpp", "result": null }
```

Most → serwer · odpowiedź OCPP · ✓ prawdziwe

```
[3, "60ca351b-c144-4e5a-b88d-b9a9e7d01a4b", { "status": "Accepted" } ] }
```

DevKit → Q11 i z powrotem · łącze szeregowe · ✓ prawdziwe, 11:56:48–50

```
11:56:48 DevKit → Q11: 55 AA 00 06 00 08 04 02 00 04 00 00 00 0A 21   ustaw punkt 4 na 10
11:56:49 Q11 → DevKit: 55 AA 03 07 00 08 04 02 00 04 00 00 00 0C 27   punkt 4 = 12, jeszcze stara
11:56:50 Q11 → DevKit: 55 AA 03 07 00 08 04 02 00 04 00 00 00 0A 25   punkt 4 = 10, przyjęta
```

DevKit → most · powiadomienie Shelly · ✓ prawdziwe

```
{ "src": "apq11dev-543204516334", "dst": "most-ocpp", "method": "NotifyStatus",
  "params": { "ts": 1790330210.70, "number:200": { "value": 10 } } }
```

- Profil mówi: „od chwili 0 limit 10 A”. TxDefaultProfile oznacza domyślny limit dla każdej transakcji, a Absolute znaczy, że czasy liczy się od wskazanej chwili.
- Most wyjmuje z profilu liczbę 10 i zamienia ją na polecenie Shelly dla pola number:200, czyli limitu prądu.
- W poleceniu Shelly "id": 8 to numer zapytania nadany przez most, a "params": { "id": 200 } to numer pola. Te dwa id nie mają ze sobą związku.
- Pole "src": "most-ocpp" mówi DevKitowi, dokąd odesłać odpowiedź i przyszłe powiadomienia.

- W ramach Tuya widać zapis 04 02 00 04 00 00 00 0A: punkt danych 4, typ liczba, 4 bajty wartości, wartość 10.

Krok 8. Zdalne zatrzymanie i koniec transakcji

Serwer → most · polecenie · 🟡 przykład

```
[2, "e5f6...", "RemoteStopTransaction", {"transactionId": 1}]
```

Most → serwer · odpowiedź · 🟡 przykład

```
[3, "e5f6...", {"status": "Accepted"}]
```

Most → DevKit → Q11 · 🟡 przykład

```
{"id": 10, "src": "most-ocpp", "method": "Boolean.Set", "params": {"id": 200, "value": false}}
DevKit → Q11: 55 AA 00 06 00 05 12 01 00 01 00 1E    ustaw punkt danych 18 na „nie”
```

Most → serwer · zapytanie · 🟡 przykład

```
[2, "f7a8...", "StopTransaction",
 {"transactionId": 1, "meterStop": 1236200,
  "timestamp": "2026-09-26T10:14:15Z", "reason": "Remote"}]
```

Serwer → most · odpowiedź · 🟡 przykład

```
[3, "f7a8...", {}]
```

- Serwer rozlicza różnicę liczników: 1 236 200 – 1 234 560 = 1 640 Wh, czyli 1,64 kWh.
- "reason": "Remote" bo zatrzymał serwer. Przy końcu ładowania przez auto byłoby Local, przy odpięciu auta EVDisconnected, przy błędzie Other.
- Potem most zgłosi StatusNotification „SuspendedEVSE”, a po odpięciu auta „Available”.

Krok 9. Odmowa i błąd

Są dwa sposoby powiedzenia „nie”.

Odmowa w zwykłej odpowiedzi. Polecenie jest zrozumiałe, ale ładowarka się nie zgadza. Na przykład serwer chce zatrzymać transakcję o innym numerze niż trwająca. 🟡 przykład:

```
← [2, "0a1b...", "RemoteStopTransaction", {"transactionId": 99}]
→ [3, "0a1b...", {"status": "Rejected"}]
```

Błąd. Ładowarka w ogóle nie obsługuje takiego polecenia, na przykład rezerwacji. Biblioteka mostu odpowiada wtedy sama, rodzajem 4. 🟡 przykład; treść opisu ustala biblioteka:

```
← [2, "1c2d...", "ReserveNow", {"connectorId": 1, "expiryDate": "...", "idTag": "KARTA-TEST", "reservationId": 5}]
→ [4, "1c2d...", "NotImplemented", "<opis od biblioteki>", {}]
```

Cała sesja na jednej osi

Krok	Nadaje → odbiera	Łączy	Rodzaj	Wiadomość
0	most → serwer	WWW, potem WebSocket	otwarcie	prośba o przełączenie, zgoda 101
1	most → serwer	WebSocket, OCPP	zapytanie	BootNotification; odpowiedź Accepted, co 60 s
2	most → serwer	WebSocket, OCPP	zapytanie	StatusNotification Available
3	most → serwer	WebSocket, OCPP	zapytanie	Heartbeat, co 60 s
4	serwer → most	WebSocket, OCPP	polecenie	RemoteStartTransaction z kartą
4	most → DevKit → Q11	WebSocket Shelly, łączy Tuya	polecenie	włącz ładowanie, punkt danych 18
5	Q11 → DevKit → most	łączy Tuya, WebSocket Shelly	meldunek, powiadomie- nie	stan: auto podłączone, ładuje
5	most → serwer	WebSocket, OCPP	zapytania	StatusNotification Charging, StartTransaction
6	most → serwer	WebSocket, OCPP	zapytanie	MeterValues, co 30 s
7	serwer → most → DevKit → Q11	wszystkie trzy	polecenie	SetChargingProfile 10 A, punkt danych 4
8	serwer → most → DevKit → Q11	wszystkie trzy	polecenie	RemoteStopTransaction, punkt danych 18
8	most → serwer	WebSocket, OCPP	zapytanie	StopTransaction z licznikiem kończącym

Zasada ogólna. Między serwerem a mostem obie strony pytają i odpowiadają w OCPP. Między mostem a DevKitem most wysyła polecenia Shelly i dostaje odpowiedzi oraz powiadomienia. Między DevKitem a Q11 idą ramki Tuya: polecenia „ustaw” i meldunki „melduję”. Most jest jedynym miejscem, które zna oba światy i pamięta numer transakcji.